

Lekcja 14: Obrazki PNG

Lekcja dodatkowa – rób ją po lekcji 10 (Bohater), zanim zaczniesz własny projekt (13).

Cel

Grafika z plików PNG zamiast literek: bohater z dwiema klatkami chodu, pszczoła machająca skrzydłami, jabłka i drzewa – wszystko narysowane w programie graficznym i wczytane jedną funkcją.

Co nowego

W lekcji 10 rysowałeś bohatera literami w kodzie. To działa, ale prawdziwe gry mają grafikę w **plikach**. Ta konsola czyta pliki **PNG** – takie same, jakie zapisuje Paint, Piskel czy Aseprite.

```
Sprite hero[2]; // tablica dwóch obrazkow = dwie klatki animacji

void setup()
{
    hero[0] = load_image("hero_0.png"); // plik assets/obrazki/hero_0.png
    hero[1] = load_image("hero_1.png");
}

void frame()
{
    int klatka = (frame_count() / 8) % 2; // co 8 klatek: 0, 1, 0, 1...
    sprite(hero[klatka], x, y, patrzy_w_lewo, 3); // rysuj 3x wiekszy, z odbiciem gdy
    trzeba
}
```

Trzy rzeczy do zapamiętania:

1. **Gdzie leżą pliki.** W katalogu `assets/<nazwa gry>/`. Nazwa gry to pierwsze słowo w `CONSOLE_ADD_GAME(...)` na końcu `gra.cpp` – tu `obrazki`, więc katalog to `assets/obrazki/`.
2. **Przezroczystość.** Tło obrazka ma być przezroczyste (w Piskelu jest takie od razu). Co jest przezroczyste w PNG, tego konsola nie rysuje. Bez półprzezroczystych, „miękkich” krawędzi – albo piksel jest, albo go nie ma.
3. **Rozmiar.** Obrazek 16×16 na ekranie 800×480 jest malutki – rysuj go powiększony (`scale = 3`). Rozmiar na ekranie to `hero[0].w * SCALE` na `hero[0].h * SCALE`.

W Scratchu duszek miał **kostiumy** i klocek „następny kostium”. Tu kostiumy to elementy tablicy `hero[2]`, a „następny kostium” to `(frame_count() / 8) % 2`.

Uruchom

1. Otwórz `gra.cpp` w tym katalogu.
2. Wciśnij **Ctrl+Shift+B** (albo **F6**).

3. Zobaczysz sad: trawa, trzy drzewa, bohater, czerwone jabłko i pszczoła latająca w prawym górnym rogu. Strzałki poruszają bohaterem w cztery strony – idąc, przebiera nogami, w lewo patrzy w lewo. Jabłka jeszcze nie da się zebrać, a pszczoła jest nieszkodliwa.

Jak to działa

- `Sprite hero[2]`, `Sprite bee[2]` – tablice klatek. `apple`, `tree` – pojedyncze obrazki.
- W `setup()` sześć wywołań `load_image(...)`. Jeśli pliku nie ma, w terminalu pojawi się komunikat, a obrazek będzie pusty (nic się nie narysuje) – gra nie wywali się.
- W `frame()`: ruch w 4 kierunkach ustawia `walking` i `facing_left`. `hero_frame` to 0, gdy bohater stoi, a `(frame_count() / 8) % 2`, gdy idzie. Pszczoła zmienia klatkę co 4 klatki i jest odbijana lustrzanie, gdy leci w lewo (`bee_dx < 0`).
- `clamp` trzyma bohatera na ekranie – pamiętaj o rozmiarze powiększonego obrazka.

Zadania

1. ★ **Własna grafika.** Wejdź na `piskelapp.com`, narysuj bohatera 16×16 (dwie klatki: nogi razem i nogi w kroku), eksportuj jako PNG (Export → PNG → „Download”, jeden plik na klatkę) i zapisz jako `hero_0.png` i `hero_1.png` w `assets/obrazki/`. Uruchom grę – kodu nie zmieniasz! Możesz też przerysować jabłko na gruszkę.
2. ★ **Większy świat.** Zmień `SCALE` na 4 i `SPEED` na 5. Co się dzieje z `clamp`? Dodaj czwarte drzewo.
3. ★★ **Zbieranie jabłek.** Gdy prostokąt bohatera nachodzi na prostokąt jabłka (`overlaps(...)` z rozmiarami pomnożonymi przez `SCALE`) – `apples++`, a jabłko skacze w losowe miejsce (`random`). (test 1)
4. ★★ **Pszczoła goni.** Zamiast patrolu: jeśli środek pszczoły jest na lewo od środka bohatera, leć w prawo, i tak samo w pionie (2 piksele na klatkę). Gdy pszczoła dotknie bohatera: `lives--`, bohater i pszczoła wracają na start. Przy `lives == 0` wyświetl „KONIEC” i pozwól zacząć od nowa klawiszem A (`restart()`). (test 2)
5. ★★★ **Pięć jabłek i drzewa jako przeszkody.** `struct Apple { int x; int y; }; Apple all_apples[5];` – zbieranie w pętli. Drzewa w tablicy `struct Tree;` po każdym kroku sprawdź funkcją `hits_tree(...)`, czy bohater wszedł w drzewo – jeśli tak, cofnij krok (zapamiętaj `old_x`, `old_y` przed ruchem). Ruch w poziomie i w pionie sprawdzaj **osobno**, wtedy bohater ślizga się wzdłuż drzewa, zamiast się zatrzymywać.

Sprawdź sam

Po zadaniu 3 bohater idący w prawo przechodzi przez jabłko:

```
console_sim.exe --game obrazki --hold RIGHT 0 200 --frames 201 --trace 50
```

W ostatniej linii `apples` większe od 0. Po zadaniu 4 stojący bohater zostaje użądłony przed 400. klatką:

```
console_sim.exe --game obrazki --frames 400 --trace 100
```

`lives` w ostatniej linii to 2 (albo mniej). Rodzic: `tools/testy.ps1 src/games/lekcje/14_obrazki`.

Dla ciekawych

- Skąd wzięły się przykładowe obrazki? Wygenerował je skrypt `tools/gen_demo_assets.py` – Python rysuje piksele i zapisuje PNG. Otwórz go i znajdź funkcję `sprite_bee`: to też program, tylko w innym języku.
- Gry Labirynt 3D i Kosmos (w menu konsoli) czytają swoje tekstury i statki dokładnie tą samą drogą (`gfx::load_png`), tylko bez warstwy dla ucznia.
- Na prawdziwej konsoli obrazki trzeba wgrać na płytkę osobno: `pio run -t uploadfs` (robi to rodzic).

Słowniczek

po polsku	w C++
wczytaj obrazek z pliku	<code>Sprite s = load_image("plik.png");</code> (w <code>setup</code> , plik w <code>assets/<id gry>/</code>)
kilka klatek animacji	<code>Sprite klatki[2]; klatki[0] = load_image("a_0.png"); ...</code>
która klatka teraz	<code>(frame_count() / 8) % 2</code>
narysuj powiększony, odbity	<code>sprite(s, x, y, odbicie, SCALE)</code>
rozmiar na ekranie	<code>s.w * SCALE, s.h * SCALE</code>
cofnij ruch, jeśli w ścianę	<code>int old_x = x; x += SPEED; if (hits_tree(...)) x = old_x;</code>